



Inventory Management App (Frontend Only)



Objective

Build a simple web app where users can:

- Add products
- View inventory
- Update quantities
- Delete items
- Persist data (using browser storage)



What Each Language Does

● HTML (Structure)

Defines **what exists** on the page:

- Forms (input fields, buttons)
- Tables (to display inventory)
- Layout containers

👉 Think of HTML as the **skeleton**

● CSS (Styling)

Controls **how everything looks**:

- Colors, spacing, fonts
- Layout (grid, flexbox)
- Responsive design

👉 Think of CSS as the **appearance**

● JavaScript (Logic)

Handles **behavior and functionality**:

- Adding/removing items
- Updating inventory
- Saving data (localStorage)
- Dynamic updates (no page reload)

👉 Think of JS as the **brain**

Project Structure

Create 3 files:

```
inventory-app/  
| — index.html  
| — style.css  
| — script.js
```

Step 1: HTML (Structure)

```
<!DOCTYPE html>  
<html lang="en">  
<head>  
  <meta charset="UTF-8">  
  <title>Inventory Manager</title>  
  <link rel="stylesheet" href="style.css">  
</head>  
<body>  
  
  <div class="container">  
    <h1>Inventory Management</h1>  
  
    <!-- Form -->  
    <form id="inventory-form">  
      <input type="text" id="name" placeholder="Product Name" required>  
      <input type="number" id="quantity" placeholder="Quantity"  
required>  
      <input type="number" id="price" placeholder="Price" required>  
      <button type="submit">Add Item</button>  
    </form>
```

```
<!-- Inventory Table -->
<table>
  <thead>
    <tr>
      <th>Name</th>
      <th>Quantity</th>
      <th>Price</th>
      <th>Actions</th>
    </tr>
  </thead>
  <tbody id="inventory-list"></tbody>
</table>
</div>

<script src="script.js"></script>
</body>
</html>
```



Step 2: CSS (Professional UI)

```
body {
  font-family: Arial, sans-serif;
  background: #f4f6f9;
  margin: 0;
}

.container {
  width: 80%;
  margin: 40px auto;
  background: white;
  padding: 20px;
  border-radius: 10px;
}

h1 {
  text-align: center;
}

form {
  display: flex;
```

```

    gap: 10px;
    margin-bottom: 20px;
  }

input, button {
  padding: 10px;
  font-size: 16px;
}

button {
  cursor: pointer;
}

table {
  width: 100%;
  border-collapse: collapse;
}

th, td {
  padding: 12px;
  border-bottom: 1px solid #ddd;
  text-align: center;
}

button.delete {
  background: red;
  color: white;
  border: none;
}

button.edit {
  background: orange;
  color: white;
  border: none;
}

```

Step 3: JavaScript (Core Logic)

```

const form = document.getElementById("inventory-form");
const list = document.getElementById("inventory-list");

```

```

let inventory = JSON.parse(localStorage.getItem("inventory")) || [];

// Render items
function renderInventory() {
  list.innerHTML = "";

  inventory.forEach((item, index) => {
    const row = document.createElement("tr");

    row.innerHTML = `
      <td>${item.name}</td>
      <td>${item.quantity}</td>
      <td>${item.price}</td>
      <td>
        <button class="edit" onclick="editItem(${index})">Edit</button>
        <button class="delete"
onclick="deleteItem(${index})">Delete</button>
      </td>
    `;

    list.appendChild(row);
  });

  localStorage.setItem("inventory", JSON.stringify(inventory));
}

// Add item
form.addEventListener("submit", function(e) {
  e.preventDefault();

  const name = document.getElementById("name").value;
  const quantity = document.getElementById("quantity").value;
  const price = document.getElementById("price").value;

  inventory.push({ name, quantity, price });

  form.reset();
  renderInventory();
});

// Delete item
function deleteItem(index) {
  inventory.splice(index, 1);
}

```

```
renderInventory();
}

// Edit item
function editItem(index) {
  const item = inventory[index];

  document.getElementById("name").value = item.name;
  document.getElementById("quantity").value = item.quantity;
  document.getElementById("price").value = item.price;

  inventory.splice(index, 1);
  renderInventory();
}

// Initial render
renderInventory();
```



Key Features Implemented

- ✓ Add products
- ✓ Edit products
- ✓ Delete products
- ✓ Persistent storage (localStorage)
- ✓ Dynamic UI updates